

AdvR 17 – Rcpp

Getting Started

Why Rcpp?

R is slow at three types of tasks:

- ❶ **Recursive problems** — each step depends on the previous (MCMC, dynamic programming)
- ❷ **Tight loops** — millions of iterations that can't be vectorised
- ❸ **Advanced data structures** — balanced trees, hash maps beyond R's built-in types

Rcpp lets you write C++ that plugs into R seamlessly:

- Typical speedups: **10–1000x** for loop-heavy code
- Handles R types (vectors, lists, data frames) natively
- Auto-generates R wrapper functions

Rule of thumb: optimise R first (vectorise, avoid copies). Use C++ only for remaining bottlenecks.

Feature	R	C++
Assignment	<code>x <- 1</code>	<code>double x = 1;</code>
Indexing	<code>x[1]</code> (1-based)	<code>x[0]</code> (0-based)
Scalar types	just numeric	<code>int</code> , <code>double</code> , <code>bool</code> , <code>String</code>
Vectors	<code>c(1, 2, 3)</code>	<code>NumericVector x = {1, 2, 3};</code>
If statement	<code>if (x > 0) {}</code>	<code>if (x > 0) {}</code> (same)
For loop	<code>for (i in 1:n) {}</code>	<code>for (int i = 0; i < n; i++) {}</code>
Function	<code>f <- function(x) {}</code>	<code>double f(double x) {}</code>
Return	implicit last value	explicit <code>return</code>
Line ending	newline	semicolon ;
Comments	<code># comment</code>	<code>// comment</code>

Every C++ statement **must** end with `;`. Forgetting it is the #1 beginner error.

cppFunction(): Inline C++

Write C++ directly in R with `Rcpp::cppFunction()`:

```
library(Rcpp)
cppFunction('
  int one() {
    return 1;
  }
')
one()
#> [1] 1
```

Key differences from R equivalent (`one <- function() 1L`):

- Must **declare** return type (`int`)
- Must use explicit `return`
- Every statement ends with `;`
- Scalars are typed: `int`, `double`, `bool`, `String`

Example: `signR()` — Scalar I/O

Scalar input/output — sign function:

```
cppFunction('
  int signR(int x) {
    if (x > 0) return 1;
    else if (x == 0) return 0;
    else return -1;
  }
')
signR(-5)
#> [1] -1
```

Note how similar this is to the R version — main differences are type declarations (`int`), semicolons, and explicit `return`.

Example: `sumC()` — Vector Input

Vector input, scalar output — sum:

```
cppFunction('
  double sumC(NumericVector x) {
    int n = x.size();
    double total = 0;
    for (int i = 0; i < n; ++i)
      total += x[i];
    return total;
  }
')
sumC(1:10)
#> [1] 55
```

Pattern: `NumericVector` accepts R vectors. Use `.size()` for length, `x[i]` for 0-based access.

Example: `pdistC()` — Vector Output

Euclidean distance from each element to a point:

```
cppFunction('
  NumericVector pdistC(double x,
                      NumericVector ys) {
    int n = ys.size();
    NumericVector out(n); // pre-allocate
    for (int i = 0; i < n; ++i)
      out[i] = sqrt(pow(ys[i] - x, 2.0));
    return out;
  }
')
```

```
pdistC(5, 1:10)
#> [1] 4 3 2 1 0 1 2 3 4 5
```

Pattern: create output vector with `NumericVector out(n)`, fill in loop, return.

C++ equivalent of R's `abs(ys - x)` — same result, but the C++ loop runs **much** faster for large vectors.

sourceCpp(): Standalone Files

For real projects, put C++ in a .cpp file:

```
// mean_c.cpp
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double meanC(NumericVector x) {
  int n = x.size();
  double total = 0;
  for (int i = 0; i < n; ++i)
    total += x[i];
  return total / n;
}
```

```
sourceCpp("mean_c.cpp")
meanC(1:10)
#> [1] 5.5
```

Key ingredients: `#include <Rcpp.h>`, `using namespace Rcpp;`, and `// [[Rcpp::export]]` before each function to expose to R.

Exercises (Getting Started)

- 1 With no arguments, `mean.default(x)` vs `meanC(x)` — which is faster for `x <- runif(1e5)`? Why?
- 2 Convert this R function to C++:

```
f4 <- function(a, b) {  
  if (a > b) a else b    # max of two values  
}
```

Solutions (Getting Started)

- 1 `meanC()` is faster: skips S3 dispatch and `na.rm` checking. `mean.default()` makes two passes (for numerical accuracy); `meanC()` makes one.

2

```
cppFunction('
  double f4C(double a, double b) {
    if (a > b) return a;
    else return b;
  }
')
f4C(3.5, 2.1)
#> [1] 3.5
```

Rcpp in Practice

R type	Rcpp scalar	Rcpp vector	Rcpp matrix
logical	bool	LogicalVector	LogicalMatrix
integer	int	IntegerVector	IntegerMatrix
double	double	NumericVector	NumericMatrix
character	String	CharacterVector	CharacterMatrix
list	—	List	—
data.frame	—	DataFrame	—

Vectors have useful methods: `.size()`, `.push_back()`, `.begin()`, `.end()`.

Use `as<type>()` to convert R objects: `as<double>(x)`.

Missing Values

Scalars: R's NA maps to type-specific sentinels in C++:

```
# Use these constants in C++:  
#   NA_REAL      (double)  
#   NA_INTEGER   (int)  
#   NA_LOGICAL   (int)  
#   NA_STRING    (String)
```

Vectors: use `NumericVector::is_na()`:

```
cppFunction('  
  LogicalVector is_naC(NumericVector x) {  
    int n = x.size();  
    LogicalVector out(n);  
    for (int i = 0; i < n; ++i)  
      out[i] = NumericVector::is_na(x[i]);  
    return out;  
  }  
' )  
is_naC(c(1, NA, 3))  
#> [1] FALSE  TRUE FALSE
```

int has no real NA — `NA_INTEGER` is just a special int value:

```
cppFunction('
  int missing_sampler() { return NA_INTEGER; }
')
missing_sampler()
#> [1] NA
```

Danger with double: `NA_REAL` is NaN, so C++ math propagates correctly. But int NA doesn't propagate — silent wrong answers:

```
# In C++: NA_INTEGER + 1 = -2147483647 (not NA!)
```

Best practice: use `NumericVector` and `double` when NAs are possible. Check with `is_na()` before arithmetic.

Rcpp sugar provides vectorised C++ equivalents of common R functions:

- **Arithmetic:** +, -, *, /, pow, sqrt, abs
- **Logical:** <, >, ==, !=, !, &, |
- **Summary:** sum(), mean(), min(), max(), range()
- **Finding values:** match(), duplicated(), which_min(), which_max()
- **d/p/q/r** for distributions: dnorm(), pnorm(), rnorm(), etc.

Sugar lets you write **vectorised** C++ that looks like R:

```
cppFunction('
  NumericVector pdistSugar(double x,
                           NumericVector ys) {
    return abs(ys - x); // Rcpp sugar!
  }
')
```

Same result as the manual `pdistC()` loop, but more readable.

- 1 Rewrite `cumsum()` in C++ as `cumsumC()`. What happens when given NA input?
- 2 Write an `any_naC()` function that returns TRUE if a `NumericVector` contains any NA.

1

```
cppFunction('
  NumericVector cumsumC(NumericVector x) {
    int n = x.size();
    NumericVector out(n);
    out[0] = x[0];
    for (int i = 1; i < n; ++i)
      out[i] = out[i - 1] + x[i];
    return out;
  }
')
```

```
cumsumC(1:5)
#> [1]  1  3  6 10 15
```

With NA input, the NA (as NaN) propagates through all subsequent sums — same behavior as R's `cumsum()`.

2

```
cppFunction('
  bool any_naC(NumericVector x) {
    int n = x.size();
    for (int i = 0; i < n; ++i)
      if (NumericVector::is_na(x[i])) return true;
    return false;
  }
')
```

`any_naC(c(1, 2, NA))`
#> [1] TRUE

STL

The **STL** provides battle-tested C++ data structures and algorithms:

- **Iterators** — generic pointers for traversing containers
- **Algorithms** — `std::sort`, `std::accumulate`, `std::unique`, ...
- **Data structures** — `vector`, `set`, `map`, `unordered_map`, ...

Why use STL with Rcpp?

- R lacks efficient ordered sets, hash maps, priority queues
- STL algorithms often faster than hand-written loops
- Rcpp converts STL containers $\leftrightarrow$ R types automatically

Iterators point to container elements: `.begin()` → first, `.end()` → past-the-last.

```
cppFunction('
  double sum_stl(NumericVector x) {
    return std::accumulate(x.begin(),
                           x.end(), 0.0);
  }
')
sum_stl(1:10)
#> [1] 55
```

Useful algorithms:

Algorithm	Does
<code>std::accumulate(b, e, init)</code>	Sum/fold
<code>std::sort(b, e)</code>	In-place sort
<code>std::upper_bound(b, e, val)</code>	Binary search
<code>std::unique(b, e)</code>	Remove adjacent duplicates
<code>std::distance(b, it)</code>	Position of iterator

Structure	R equivalent	Use case
<code>vector<T></code>	<code>c()</code>	Growable array
<code>set<T></code>	<code>unique(sort(x))</code>	Unique sorted elements
<code>unordered_set<T></code>	(no direct)	Fast membership testing
<code>map<K,V></code>	<code>names(x)</code>	Key-value (sorted)
<code>unordered_map<K,V></code>	environment	Key-value (hashed, fast)

Example: set for unique sorted values:

```
cppFunction('
  NumericVector uniqueC(NumericVector x) {
    std::set<double> seen;
    for (int i = 0; i < x.size(); ++i)
      seen.insert(x[i]);
    return Rcpp::wrap(
      std::vector<double>(seen.begin(),
                          seen.end()));
  }
')
```

Case Study: `tableC()` with `unordered_map`

Reimplement `table()` using a hash map:

```
cppFunction('
  std::map<int, int> tableC(IntegerVector x) {
    std::map<int, int> counts;
    for (int i = 0; i < x.size(); ++i)
      counts[x[i]]++;
    return counts;
  }
')
tableC(c(1L, 2L, 1L, 3L, 2L, 1L))
#> 1 2 3
#> 3 2 1
```

Pattern: `counts[key]++` auto-initialises to 0 on first access.

For large inputs, `unordered_map` is $O(1)$ lookup vs `map`'s $O(\log n)$.

Case Study: `rleC()`

Run-length encoding — classic loop that R can't vectorise:

```
cppFunction('
  List rleC(NumericVector x) {
    int n = x.size();
    std::vector<int> lengths;
    std::vector<double> values;
    int i = 0;
    while (i < n) {
      int j = i + 1;
      while (j < n && x[j] == x[i]) j++;
      lengths.push_back(j - i);
      values.push_back(x[i]);
      i = j;
    }
    return List::create(
      _["lengths"] = lengths,
      _["values"] = values);
  }
')
```

```
rleC(c(1, 1, 2, 2, 2, 3))
```

- 1 `median()` is slow for large vectors. Implement a `medianC()` using `std::nth_element()` (partial sort, $O(n)$ instead of full $O(n \log n)$ sort).
- 2 Implement `duplicatedC()` that returns a `LogicalVector` indicating which elements have been seen before. Use `unordered_set`.

1

```
cppFunction('
    double medianC(NumericVector x) {
        int n = x.size();
        NumericVector y = clone(x); // dont modify input
        std::nth_element(y.begin(),
            y.begin() + n/2, y.end());
        if (n % 2 == 1) return y[n/2];
        std::nth_element(y.begin(),
            y.begin() + n/2 - 1, y.end());
        return (y[n/2 - 1] + y[n/2]) / 2.0;
    }
')
```

2

```
cppFunction('
  LogicalVector duplicatedC(IntegerVector x) {
    int n = x.size();
    LogicalVector out(n);
    std::unordered_set<int> seen;
    for (int i = 0; i < n; ++i) {
      if (seen.count(x[i])) out[i] = true;
      else { out[i] = false; seen.insert(x[i]); }
    }
    return out;
  }
')
```

```
duplicatedC(c(1L, 2L, 1L, 3L, 2L))
#> [1] FALSE FALSE  TRUE FALSE  TRUE
```